

swtpcsim — The Debugger

2026-09-27 · 0e4d89e

*A simulation of the SWTPC 6800 on the SS-50/SS-30 bus, and the MITS Altair
680b*

Debugging	1
Where the processor is — REGS	1
Stepping — STEP , NEXT	2
Breakpoints — BREAK , NOBREAK	2
Reading a block of memory — DUMP	4
One byte at a time — EXAMINE , DEPOSIT , EDIT	5
Disassembling — DISASM	6
Symbols — SYMBOLS , SHOW SYMBOLS	8
Searching, filling, moving — SEARCH , FILL , MOVE , COMPARE	9
Reaching a device register by hand — EXAMINE , DEPOSIT	9
Asking without touching — WHO	9
Looking at the bus itself — SHOW BUS	10
The machine over time — TRACE , HISTORY	11
What a board is doing — SET ... DEBUG , SHOW DEBUG	13
A copy of the session — SET CONSOLE log	15
A debugging session	15
Saving state — SNAPSHOT , RESTORE	16
Getting an assistant to do it — the MCP server	17
Things to know about the bus	18

Debugging

This is the document the simulator exists for.

It is the companion to *The Monitor*: that document introduces the `swtpcsim>` prompt — how you start, stop and reconfigure the machine — and this one covers what you do at that prompt when something has gone wrong. Both stand beside the *User Manual*, which describes the emulated hardware. `swtpcsim` simulates Motorola 6800 machines — the SWTPC 6800 on the SS-50/SS-30 bus, and the MITS Altair 680b; if the `swtpcsim>` prompt is new to you, read *The Monitor* first.

Running old software is the easy half. The hard half is being able to see what a machine is actually *doing* — which board answered, what went out on the bus, why the interrupt never arrived — and that is what the commands here are for. Not every monitor command is one of them — these are the ones you reach for when something has gone wrong.

Where the processor is — REGS

`REGS` prints the whole processor on one line. The flags come first — half-carry, interrupt mask, negative, zero, overflow, carry — then the two accumulators, the index register, the stack pointer, and the program counter. The last column is **the instruction the processor is about to execute**, already disassembled.

You get this line free every time the machine stops, so most of the time you never type `REGS` at all.

```
swtpcsim> REGS
H0I1N0Z0V0C0 A=02 B=10 X=8004 SP=A03D PC=E201 ASRA
```

The flags are the 6800's condition codes, in the order the processor numbers them — `H` half-carry, `I` the interrupt mask, `N` negative, `Z` zero, `V` overflow, `C` carry — each shown as its bit. Then the two 8-bit accumulators `A` and `B`, the 16-bit index register `X`, the stack pointer `SP`, and the program counter `PC`. Each register is labelled the way you would name it to `SET REG`, so what you read is what you type back.

The line follows the CPU in the machine, so it shows exactly the registers that processor has. It is a 6800 here — a flat 64K, big-endian machine whose I/O is memory-mapped, so there is no separate bank of ports to print and no privileged register beyond the ones above.

Flags are registers, and you may set them:

```
SET REG A=3F
SET REG C=1
```

Stepping — STEP, NEXT

STEP runs **real bus cycles through the real instruction decode**. It is not an interpreter running alongside the machine — it is the machine, moved forward by one instruction. It prints one line per instruction — the machine *after* that instruction ran, with the one the PC has now reached — so **STEP 3** shows three lines, one for each step. Past thirty-two it runs quietly and tells you where it ended up.

```
STEP      one instruction
STEP 20   twenty of them (a count, so it is decimal)
```

NEXT **steps over a subroutine**. At a **JSR** or **BSR**, **STEP** walks you down into the callee — every instruction it runs, and everything it in turn calls. Often you do not care: the routine works, and you want the *next* instruction in the code you are reading, not a tour of a print routine. **NEXT** gives you that. On a **JSR** or **BSR** it runs the callee at full speed and stops the instant it returns; on anything else it is just a single step. It does exactly what you would do by hand — sets a breakpoint at the return address and runs to it — so the callee is live while it runs: it can read the console, and **^E** (STOP) or **^C** stops it if it never comes back. A breakpoint that fires *inside* the callee stops you there, as it should.

```
NEXT      over the JSR/BSR at PC, else one instruction
N         the same -- it owns the letter, because you type it constantly
```

Breakpoints — BREAK, NOBREAK

There are three kinds, and only the first is about the processor at all.

BREAK MEM **watches bus cycles, not instructions**. That is a much stronger thing, and it is the reason to prefer it. A memory watch will catch a write no instruction on the CPU appeared to make, and it will work unchanged on any processor you put in the machine, because it is watching the backplane rather than the program. And on a 6800 the I/O is memory: the console ACIA answers at addresses, not ports, so **BREAK MEM R 8005** catches every read of the console's receive register, and **BREAK MEM W 8005** every byte sent to its transmitter — the memory watch is how you watch a device, not just RAM.

If you are chasing a byte that keeps getting clobbered, **BREAK MEM W <addr>** will find who is doing it, whatever is doing it.

A cycle watch stops *before* the access, with the PC on the instruction that was about to make it — nothing executed, no byte read, no byte written, every register as it stood. It is the same place a plain **BREAK <addr>** stops, so **RUN** or **STEP** runs that instruction fresh.

The 6800 is **memory-mapped** — a UART, a floppy controller, a PIA is an address, not a port — so a register access is a memory cycle, and `BREAK MEM R|W <addr>` on a device register catches it. There is no separate I/O watch because there is no I/O port space.

BREAK TAPE STOP watches a device, not the program at all. It stops the machine the moment a cassette deck reaches auto-stop — the instant the tape parks itself after a load has finished feeding. That is exactly when you want to look at what landed, and you get there without having to know the loader's end address: arm it, run, and the machine halts inside the loader the moment the tape stops. (It applies where there is a cassette to watch — the Altair 680b with its KCACR Kansas-City interface.)

```
BREAK E101          stop when the PC gets there
BREAK 2C00-2CFF     ...or anywhere in a range
BREAK MEM W A043    stop when ANYTHING writes to A043
BREAK MEM R 8005    stop on a read of the console receive register
BREAK TAPE STOP     stop when a cassette deck auto-stops after a load
BREAK              list them
NOBREAK 2           clear one (the id is a plain decimal, not a bus address)
NOBREAK            clear them all
```

Breakpoint ids are handed out in order — 1, 2, 3 — and restart at 1 once no breakpoints are left, whether you cleared them all with `NOBREAK` or removed the last one by id. Removing a breakpoint from the middle does not renumber the rest, so an id is not the running count of live breakpoints.

When one fires, it says which, where, and what the machine was doing. A plain address breakpoint stops with the PC *on* the instruction it names — nothing there has run yet:

```
swtpcsim> BREAK E101
breakpoint 1: pc      E101
swtpcsim> RUN E0D0
breakpoint 1 (pc      E101) -- stopped at E101
2 instructions, 7 cycles.
H0I1N1Z0V0C0 A=02 B=10 X=8004 SP=A042 PC=E101 STS A008
```

The header names the breakpoint that fired and where the machine stopped, then how far it ran since `RUN`, then the register line `REGS` would print — flags, registers, `PC` (the program counter), and the instruction about to run. A cycle watch reads the same way, but *stopped at* is the instruction that made the

access, not the watched address — so it hands you the culprit. Here a write to `A043` is caught with the PC on the `STAA A043` that wrote it, five instructions into SWTBUG's reset:

```
swt pcsim> BREAK MEM W A043
breakpoint 1: mem w A043
swt pcsim> RUN E0D0
breakpoint 1 (mem w A043) -- stopped at E308
5 instructions, 24 cycles.
H0I1N1Z0V0C0 A=FF B=10 X=8004 SP=A040 PC=E308 STAA A043
```

An address breakpoint can carry a condition. `BREAK <addr> IF <expr>` stops only when the expression is true — the registers, tested the moment the PC reaches the address. It is what you reach for when a breakpoint fires ten thousand times before the once you care about: put the distinguishing state in the condition and let the machine run until it holds.

A bare word that names a register *is* that register, so a literal needs a leading zero — `0A` is ten, `A` is the accumulator. `==` `!=` `<` `>` `<=` `>=` compare, `&&` `||` combine, `&` `|` mask, and parentheses group.

```
BREAK E101 IF A==0
BREAK E101 IF X==8004 && Z==1
BREAK E101 IF (A&0F)==0      only when the low nibble is zero
```

A cycle watch can carry a condition too. `BREAK MEM W A043 IF <expr>` stops only on the access whose registers satisfy the condition — so you can wait for the *one* write to a shared byte that happens while a particular register holds. The registers `IF` tests here are the ones the instruction ran *with* — its inputs, as they stood the moment it began — the same state a `BREAK <addr> IF` at that instruction would see.

```
BREAK MEM W A043 IF B==0      the write to A043 taken with B already zero
```

LOADS tests what a read fetched. An `IF` on a read sees the registers *before* the instruction, so it cannot ask about the byte just fetched — that byte is not in any register yet. `LOADS <expr>` is the other half: it judges the condition *after* the instruction retires, so the register the read loaded holds its new value. It belongs to a read — `BREAK MEM R <addr> LOADS <expr>` — because a write loads no register into which the condition could look, so `LOADS` is only accepted there. On the memory-mapped 6800 that read is often of a device register: `BREAK MEM R 8005 LOADS A>7F` stops only when the ACIA's data register hands back a byte over `7F`.

Reading a block of memory — `DUMP`

`DUMP` is how you read a lot of memory at once. A bare `DUMP <addr>` runs to the **end of its page**, and a bare `DUMP` carries on from there — so however you first landed, the rows stay page-aligned and the columns never move under your eye.

It only *looks*. Nothing is consumed and no bus cycle is run.

```
DUMP 100          0100-01FF: a whole page
DUMP              the next page
DUMP E000-E00F    exactly that range
DUMP 100/20       0100-011F (a length, and it is part of the address, so it is hex)
DUMP 0 WIDTH=8    eight bytes to a line (a count: decimal)
```

Each row is the address, then the bytes in hex, then the same bytes as text — a byte that is not a printable character shows as `.`. Here is the start of the SWTBUG ROM at `E000`:

```
swtpcsim> DUMP E000-E03F
E000  FE A0 00 6E 00 8D 40 6E  00 10 16 04 BD E3 34 8D  ...n..@n.....4.
E010  67 81 53 26 FA 8D 61 81  39 27 29 81 31 26 F0 7F  g.S&..a.9').1&..
E020  A0 0F 8D 31 80 02 B7 A0  47 8D 1C 8D 28 7A A0 47  ...1....G...(z.G
E030  27 09 A7 00 A1 00 26 08  08 20 F0 7C A0 0F 27 CF  '.....&.. .|...'
```

This is code, so the text column is mostly `.` — it earns its keep on a buffer of strings, where you can read the message straight out of the right-hand column.

One byte at a time — EXAMINE , DEPOSIT , EDIT

These do by hand what a front panel's switches did — on the Altair 680b that was a real panel of switches and lamps, and the SWTPC had none, so on it these commands *are* that panel. `EXAMINE` shows a single byte — hex, ASCII, and its bits. It also jams the address into the program counter, exactly as the switch did, so `EXAMINE <addr>` follows the byte with the register line and the disassembled instruction the PC now points at — what the next `STEP` will run. A bare `EXAMINE` steps to the next byte, quietly, which is the panel's `EXAMINE NEXT`.

```
swtpcsim> EXAMINE E0D0
E0D0  8E . 10001110
H0I1N0Z0V0C0 A=02 B=10 X=8004 SP=A03D PC=E0D0 LDS #A042
```

`DEPOSIT` runs a **real bus write**. If no board decodes that address, it says so rather than pretending to have stored something — which is the difference between a debugger and a notepad.

```
EXAMINE E0D0      one byte: hex, ASCII, and its bits -- then the register line
                  and the instruction the PC now points at, ready for STEP
EXAMINE           the next byte, quietly -- the panel's EXAMINE NEXT
DEPOSIT 100 7E E0 D0
```

`EDIT` is `DEPOSIT` done interactively — the way you patch a run of bytes without retyping the address each time. The prompt shows an address and the byte that is there; type a new value and Enter writes it and drops to the next byte, a bare Enter leaves the byte untouched and drops to the next, and `.` returns you to the monitor. It is the same real bus write, so it warns the same way when nothing decodes the address, and `EDIT <addr> ROM` burns a PROM. `EDIT` needs a keyboard — at the prompt or down a pipe — so where there is none (an automated `startup` list) reach for `DEPOSIT` instead.

```
EDIT 100      0100 C3 7E      type 7E, Enter – written, on to 0101
              0101 00      Enter alone – left as 00, on to 0102
              0102 2C .      a '.' stops and returns to the monitor
```

On a machine with a CPU, `EDIT` will also take an **instruction** where a byte would go and assemble it in place — `EDIT` is `DISASM` (below) read the other way. Type `LDAA #FF` and it writes `86 FF`; the prompt then drops by the instruction's length, not one byte, so a two-byte instruction lands the next prompt two on. Operands are numbers in the console base — an `H` or `Q` suffix on the number overrides it — and there are no labels: this is a patch assembler, not a toolchain. A bare value is still a plain byte, so byte entry is unchanged. The 6800 assembles in full; a CPU whose encoding is not assembled here keeps taking bytes.

```
EDIT 100      0100 86 LDAA #FF    assembles 86 FF, on to 0102
              0102 00 JSR E057    assembles BD E0 57, on to 0105
              0105 76 .          '.' returns to the monitor
```

Disassembling — `DISASM`

`DISASM` **peeks**: it reads memory without running a bus cycle. That matters, and it is not a detail. A `read()` on a serial board *consumes* a byte from its receiver, and a disassembler that ate the guest's input while you were looking at it would be a debugger you could not trust. Nothing here that only *looks* at memory will disturb it.

```
DISASM E0D0    sixteen instructions
DISASM        carry on
DISASM 0-2F    exactly that range
```

A worked example — SWTBUG's RESET entry at `E0D0` , the first thing the ROM runs:

```
swtpcsim> DISASM E0D0-E0EF
E0D0  8E A0 42  LDS #A042
E0D3  20 2C     BRA E101
E0D5  26 07     BNE E0DE
E0D7  09        DEX
E0D8  09        DEX
E0D9  FF A0 0D  STX A00D
E0DC  20 AC     BRA E08A
E0DE  FF A0 0D  STX A00D
E0E1  20 02     BRA E0E5
E0E3  20 6D     BRA E152
E0E5  81 30     CMPA #30
E0E7  25 A1     BCS E08A
E0E9  81 46     CMPA #46
E0EB  22 9D     BHI E08A
E0ED  8D BD     BSR E0AC
E0EF  BD E0 57  JSR E057
```

It loads the stack pointer (`LDS #A042` — SWTBUG keeps its stack in the scratchpad RAM at `A000`) and branches straight over the command-dispatch table that follows into the real start-up at `E101` . Those middle lines — the `CMPA #30` / `CMPA #46` that fence a hex digit, the `BSR` / `JSR` that call subroutines — are the command decoder the branch skips on the way in.

DISASM trusts you to start on an opcode, and it cannot check. Give it an address in the *middle* of an instruction and it will decode the operand bytes as if they were opcodes, and the listing it prints is fiction. Start the same reset code one byte late, at `E0D1` , and the `A0 42` that was the *operand* of `LDS #A042` becomes an instruction in its own right:

```
swtpcsim> DISASM E0D1-E0DF
E0D1  A0 42     SUBA 42,X
E0D3  20 2C     BRA E101
E0D5  26 07     BNE E0DE
...
```

`SUBA 42,X` is a phantom — there is no such instruction in this ROM. A disassembler usually re-syncs after a byte or two (here `E0D3` is already back on the real `BRA E101` , because the stray `A0 42` happened to be two bytes long), so a listing can look right a few lines down while its first instruction is nonsense. When a **DISASM** reads oddly, check that you started where an instruction *starts*: single-step to the address with `STEP` , or begin the range at a label you trust.

DISASM decodes for the CPU the machine is running, and prints in that CPU's own assembly dialect. Both built-in machines are 6800s, so it reads Motorola mnemonics — `LDAA` , `STAA` , `JSR` , `BRA` ; put a

different processor in the machine and the same bytes would decode as that processor's instructions instead.

Symbols — SYMBOLS, SHOW SYMBOLS

Everything so far has spoken in hex. Load an assembler's symbols and you can name things instead: `BREAK START` rather than `BREAK 0100`, `DUMP MSG/20`, `EXAMINE OUTCH`. A symbol is accepted anywhere an address is typed, and in a `BREAK ... IF` condition.

<code>SYMBOLS LOAD prog.SYM</code>	a symbol table
<code>SYMBOLS LOAD prog.PRN</code>	...or an assembler listing
<code>BREAK START</code>	
<code>DUMP MSG/20</code>	
<code>BREAK 200 IF X==STACK</code>	
<code>SHOW SYMBOLS</code>	all of them
<code>SHOW SYMBOLS OUT*</code>	filtered by a glob
<code>SYMBOLS CLEAR</code>	forget them

With a table loaded, `DISASM` reads symbolically. A program **label** heads its own line the way an assembler listing prints it, so a jump destination announces itself where it lands; and a 16-bit **operand** reads as a name, so `JSR E057` becomes `JSR OUTCH` when `E057` is named. Two things use the table differently, and the difference is deliberate. A leading label comes from **program labels only**: a constant that merely equals a code address must not masquerade as one, so an `EQU` never *heads* a line. But an operand is a value the instruction *points at*, and there an `EQU` that is really an address is exactly what you want to read — so an `EQU` naming an I/O register is offered back as the operand's name. A real label wins when a label and an `EQU` share a value. Only a 16-bit operand is treated as an address; a byte immediate stays a number, because two hex digits are a count, not an address.

`DISASM` is the only command whose *output* is symbolic: `DUMP` still prints hex and ASCII, since nothing in a data block says which bytes are an address and which are just bytes.

Two kinds of file, and the toolchains that write them. A `.SYM` is a flat list of name = value; a `.PRN` or `.LST` is an assembler's own listing. The loader reads the Microsoft `M80` listing (`.PRN`) and `L80` symbol table (`.SYM`) formats — decided by the file's content, not its extension. The listing is the richer source, because it marks an `EQU` and so can tell a constant apart from a program label: only real labels are offered back as addresses. An `L80 .SYM` holds **globals only** (the `PUBLIC` names), so a module's local labels and `EQU`s are not in it; for those, use the listing. (`docs/sources.md` records the toolchain manuals kept for this feature.)

Addresses must be absolute. A relocatable `M80` listing marks its addresses, and loading one is refused by the offending line — link it and load the `.SYM`, or assemble to an absolute origin. A `.SYM` is written after linking and is absolute already, so it never has this problem.

Symbols are yours, not the machine's. Like a breakpoint, the table is the debugger's view, not part of any board — it survives `RESET`, `POWER`, and `CONFIG LOAD`, and `SYMBOLS CLEAR` is its `NOBREAK`. Loading two files **merges** them (the newest of a clashing name wins, and the command says how many were redefined); `SYMBOLS LOAD <file> REPLACE` starts fresh. A machine file can name a symbol file in its `startup`, and `CONFIG SAVE` writes the filename back out — the file, not the parsed table, exactly as it does for a built-in ROM.

A name beats a hex literal. If a symbol is spelled like a number — `FACE`, `BEEF` — the symbol wins; write `0FACE` (or `$FACE`) to force the number, the same escape that tells the register `A` from the number `0A`.

Searching, filling, moving — `SEARCH`, `FILL`, `MOVE`, `COMPARE`

The block operations. `COMPARE` will take a file as its second operand, which is how you check what the machine loaded against what you meant to load.

```
SEARCH E000-E3FF 8E A0      find those bytes
SEARCH 0-FFFF "SWTBUG"     ...or that string
FILL 100-1FF 00
MOVE 100-1FF 2000
COMPARE 100-1FF 2000       ...or against a file
```

Reaching a device register by hand — `EXAMINE`, `DEPOSIT`

The 6800 is **memory-mapped**: a device's registers *are* memory, so you reach them the same way you reach RAM. `EXAMINE <addr>` runs a real read cycle and `DEPOSIT <addr> <byte>` a real write cycle, with every side effect the guest's own access would have — an `EXAMINE` of a UART's data register *consumes* the byte just as the guest would. There is no `IN`/`OUT`, because there is no port space; a UART at `8005` is poked exactly like a byte of RAM.

Asking without touching — `WHO`

`WHO` asks which **board** *would* answer — the one that decodes the address you name, reported by its board id. **No cycle is run and nothing is consumed.** It is the question you want when a read gives you `FF` and you cannot tell whether that is data or whether nothing is there at all.

The console ACIA decodes `8004`, so it answers by name — and reads and writes can land on different boards, so `WHO` reports each:

```
swtpcsim> WHO 8004
8004 read mps0
8004 write mps0
```

An address nothing decodes answers `nobody`, and now you know an `FF` there was a floating bus, not data:

```
swtpcsim> WHO 9000
9000 read nobody -- floats to FF
9000 write nobody -- floats to FF (a write here is simply gone)
```

It reports contention, so if two boards are fighting over an address, `WHO` is where you find out. `WHO <addr>` asks about any address — and since I/O is memory-mapped, that includes every device register.

Looking at the bus itself — `SHOW BUS`

Where `WHO` asks about one address, `SHOW BUS` shows you the whole backplane at once.

`SHOW BUS MAP` is the one you reach for most: it lays out who decodes what across the whole address space, and what is left floating. On the `swtpc` machine the console ACIA, the DC-4 floppy latch and registers, the RAM and the SWTBUG ROM each claim their range, and the gaps between are named:

```
swtpcsim> SHOW BUS MAP
MEMORY
  0000-7FFF mem0 ram
  8004-8007 mps0 read/write 6850 ACIA 'tty' -- A0 selects: base/base+2 status/control, base+1/base+3
Rx/Tx data
  8014-8014 dc40 drive select D0-D1 drive 0-3, D6 side (write-only, true-sense)
  8018-8018 dc40 command/status WD179x
  8019-8019 dc40 track WD179x
  801A-801A dc40 sector WD179x
  801B-801B dc40 data WD179x
  A000-BFFF mem0 ram
  E000-E3FF mem0 rom builtin:swtbug
  FC00-FFFF mem0 rom builtin:swtbug
unmapped: 8100-9FFF,C000-DFFF,E400-FBFF (floats to FF)
```

Since I/O is memory-mapped, a device's registers appear in this same map — the SWTBUG console ACIA at `8004 – 8005`, the DC-4 floppy latch and WD179x above it — so there is no separate I/O map to print.

`SHOW BUS CONTENTION` is the one to reach for when a machine you built yourself is misbehaving for no reason. Two boards decoding the same address is a real hardware fault, and the simulator will not quietly pick a winner for you; with none, it says so:

```
swt pcsim> SHOW BUS CONTENTION
none.
```

`SHOW BUS IRQ` is the window onto the interrupt wiring, and interrupt wiring is the part of a machine you cannot otherwise see. The 6800 takes its interrupts through vectors at the very top of memory — `IRQ` at `FFF8`, the `SWI` software trap at `FFFA`, `NMI` at `FFFC`, and `RESET` itself at `FFFE` — and the `I` flag in `REGS` masks the maskable one. A board strapped to pull a line that the software never enables fails in total silence; this command is what makes the wiring visible.

```
SHOW BUS MAP      who decodes what in memory -- and what floats
SHOW BUS IRQ      the interrupt wiring: who is strapped to pull a line, and who is pulling now
SHOW BUS CONTENTION where two boards answer the same address
```

The machine over time — `TRACE`, `HISTORY`

`WHO` and `SHOW BUS` are snapshots — the backplane as it is *now*. `REGS` and `STEP` are the machine as it is *now*. `TRACE` and `HISTORY` show you all of that over *time*, which is what you want when the bug is not where the machine stopped but somewhere in how it got there.

`HISTORY` is a flight recorder. A fixed-size ring is always filling while the machine runs, so when a breakpoint fires — or the machine wanders off into the weeds — the run-up to it is *already* recorded. You do not arm it; it is on. A bare `HISTORY` shows the last sixteen **instructions**, oldest first; `HISTORY <n>` shows the last *n*. Each line is exactly what `STEP` prints — the registers and flags as the machine stood, and the decoded mnemonic it was about to run — so the recording reads like a `STEP` you did not have to be there for. The mnemonic is decoded from the bytes that *actually ran* at that address, so self-modifying code reads truthfully.

There is a second recorder underneath, for when the CPU view is not enough: **`HISTORY BUS`** is the raw bus cycles — no registers and no mnemonics, just `CYCLE`, `TYPE`, `ADDR` and `DATA`, and then **who drove the cycle and who answered it**. The processor drives the cycles, so that column reads `cpu`; the *answered* column is the board that decoded the address — or `--` when nobody did and the read floated to `FF`.

Here is SWTPBUG polling its console ACIA — instruction fetches from the ROM answered by `mem0`, and the status read at `8004` answered by `mps0`:

```
swtpcsim> HISTORY BUS 6
CYCLE  TYPE ADDR DATA    DROVE    -> ANSWERED
 49144  MR   E201 = 47   cpu      -> mem0
 49146  MR   E202 = 24   cpu      -> mem0
 49146  MR   E203 = FB   cpu      -> mem0
 49150  MR   E1FF = A6   cpu      -> mem0
 49150  MR   E200 = 00   cpu      -> mem0
 49150  MR   8004 = 02   cpu      -> mps0
```

```
HISTORY          the last 16 instructions
HISTORY 100      the last hundred (a count, so decimal)
HISTORY BUS      the last 16 bus cycles instead
HISTORY BUS 100  the last hundred cycles
```

TRACE logs every cycle as it happens — to the console, or to a file. Like the breakpoints that watch the bus, it is not a CPU feature: it watches the same stream every board sees, so it works unchanged on any processor you put in the machine.

A **MASK** narrows the log to the kinds of cycle you name. With no mask every cycle is logged; with one, a cycle is kept if it matches *any* kind you listed. The two kinds are **IRQ** (an interrupt-acknowledge cycle) and **CONTENTION** (a cycle more than one board answered — the fault **SHOW BUS CONTENTION** reports, caught as it happens). A 6800 has no interrupt-acknowledge bus cycle — it vectors through fixed memory with an ordinary read — so on `swtpc` or `altair680` **CONTENTION** is the filter that catches anything; otherwise you trace all cycles, or narrow by *place* with a tracepoint (below), rather than by category.

```
TRACE ON          every cycle, to the console
TRACE ON run.log   ...to a file instead
TRACE ON MASK=CONTENTION just the bus faults
TRACE OFF         stop tracing
```

Tracepoints — tracing one part of a program

A whole-program trace is a firehose. A mask narrows it by *category*, but often you do not want a category — you want a *place*: this subroutine, and nothing else.

That is a tracepoint. Add **TRACE ON** or **TRACE OFF** to a **BREAK** and it stops being a breakpoint: instead of stopping the machine it flips the trace, and the machine runs on. Two of them bracket a region.

```
swtpcsim> BREAK E101 TRACE ON    start tracing when PC reaches E101
swtpcsim> BREAK E152 TRACE OFF   ...and stop again at E152
swtpcsim> RUN E0D0
```

`TRACE ON` traces the instruction *at* its address; `TRACE OFF` does not. The region is `[on, off)` — exactly the half-open range you would write down if someone asked you which instructions were in the subroutine.

A trace toggle works on the bus kinds the same way `IF` does, and the cycle that triggered it is the *first line* of the trace, not the line above it: a trace should show its own reason.

```
swtpcsim> BREAK MEM W A043 TRACE ON      trace onward from whatever writes A043
```

They compose with `IF`, and they still do not stop:

```
swtpcsim> BREAK E101 IF X==8004 TRACE ON
```

Where the trace goes is `TRACE`'s business, not the tracepoint's. A tracepoint that has never been told anything traces to the console. To send it to a file, configure it first — and this is what `TRACE OFF` is for: it stops the tracing but *remembers where it was going*, so a tracepoint can pick it up again.

```
swtpcsim> TRACE ON run.log               configure it: file -- and it starts
swtpcsim> TRACE OFF                     stop, but the file is remembered
swtpcsim> BREAK E101 TRACE ON           arm the region
swtpcsim> BREAK E152 TRACE OFF
swtpcsim> RUN E0D0                      run.log gets the cycles from E101 to E152, and nothing else
```

Tracepoints appear in `BREAK`'s listing with the rest, and their `hits` count the times they fired. A tracepoint never hides an ordinary breakpoint at the same address: if both are set, the trace flips *and* the machine stops.

What a board is doing — `SET ... DEBUG`, `SHOW DEBUG`

`TRACE` and `HISTORY` watch the *bus* — the cycles, addresses and data every board shares. Some parts of the machine can also narrate what they are doing in their *own* terms: a floppy controller stepping the heads and reading a sector, a serial chip taking a byte, a socket answering a call. That is what the **diagnostic channels** are for. Each instrumented part has a named channel with a handful of flags, and you switch on the ones you want to hear about.

`SHOW DEBUG` lists every channel there is, its flags, and where the output is going:

```
swtpcsim> SHOW DEBUG
debug  (runtime diagnostics -- the sink and flags do not survive CONFIG SAVE)

sink  stderr  -- SET CONSOLE DEBUG=stderr|stdout|<file>

CHANNEL  FLAGS  (an enabled flag is UPPER-CASE)
-----  -----
socket   connect
6850     serial

SET <channel> DEBUG=<flag>[,<flag>] enables; NODEBUG=<flag> disables;
DEBUG=all / DEBUG=none turn every flag on / off.
```

The channels a machine offers depend on the boards in it: the `6850` serial chip and the `socket` layer are always there, and a machine with a DC-4 floppy adds a disk channel (`dsk0`) that narrates seeks and sector reads. A flag printed in capitals is on. Turn one on with `DEBUG=`, off with `NODEBUG=`; both are additive, take a comma-separated list, and understand `all` and `none`:

```
SET 6850 DEBUG=serial      narrate every byte the console ACIA moves
SET dsk0 DEBUG=sector,seek ...both sector reads and head seeks, on a disk machine
SET dsk0 DEBUG=all        everything that board can say
SET dsk0 NODEBUG=all       silence it
```

The name in front of `DEBUG` is the channel. Usually it is a board's id, but a shared chip or the socket layer has one too (`6850`, `socket`), so the same switch reaches parts of the machine that are not boards at all. An unknown flag is refused and *nothing* changes: the switch is all-or-nothing, so a typo in a list never leaves half of it applied.

Every line names its channel and is prefixed with **the PC of the instruction that drove it**, so a diagnostic line points straight at the code working the board:

```
2C38 dsk0: sector drive=0 track=0 sector=1
007F dsk0: seek drive=0 track=0 -> 1
```

At the monitor prompt — with the machine stopped — there is no such instruction, and the column reads `----`.

Where the output goes is one setting for the whole facility, aimed with `SET CONSOLE DEBUG=`:

```
SET CONSOLE DEBUG=stderr    the default
SET CONSOLE DEBUG=stdout
SET CONSOLE DEBUG=trace.log  append to a file
```

Tab completes all of it — the channel names after `SET`, `DEBUG` / `NODEBUG` after the channel, and the flag values (with `all` and `none`) after the `=`.

None of this is saved by `CONFIG SAVE`. A diagnostic is something you switch on to watch a problem, not a property of the machine, so a config you write while debugging does not carry the noise into every later run.

A copy of the session — `SET CONSOLE log`

`TRACE` records the bus and `DEBUG` records what a board narrates; `SET CONSOLE log` records the *terminal* — everything you saw. Guest output and the keys you typed both go to a host file as they happen, so you can read a whole session back later, or hand it to someone who was not there.

```
SET CONSOLE log=session.txt      start copying the session to a file
SET CONSOLE log=off              stop (an empty path does the same)
```

The file is opened for *append*: pointing `log` at the same file twice in a session adds to it rather than erasing what you already caught. Like `TRACE` and `DEBUG` it is a diagnostic and not part of the machine, so `CONFIG SAVE` does not carry it — a config written while you are capturing a session does not turn logging on for every later run.

A debugging session

Here is the whole toolset in one short session on the `swtpc` machine, watching SWTBUG bring itself up at reset. Suppose you want to know what writes the scratchpad byte at `A043` — set a memory-write watch there and run from the reset entry:

```
swtpcsim> BREAK MEM W A043
breakpoint 1: mem w A043
swtpcsim> RUN E0D0

breakpoint 1 (mem w A043) -- stopped at E308
5 instructions, 24 cycles.
H0I1N1Z0V0C0 A=FF B=10 X=8004 SP=A040 PC=E308 STAA A043
```

The watch found the culprit without your knowing the address ahead of time: the PC is on the `STAA A043` that made the write, `A` holds the `FF` about to land there, and the header says it took five instructions to get here. Disassemble to see what this routine is:

```
swtpcsim> DISASM
E308 B7 A0 43 STAA A043
E30B FE A0 12 LDX A012
E30E 8C E1 23 CPX #E123
E311 27 06 BEQ E319
E313 CE E1 24 LDX #E124
E316 FF A0 12 STX A012
E319 39 RTS
```

It stores `A` into the flag at `A043`, then loads a pointer from `A012`, and if that pointer is still at its power-on value (`E123`) leaves it, otherwise resets it to `E124` — `SWTBUG` fixing up its command-input vector. Step through it and watch the pointer settle:

```
swtpcsim> STEP 8
H0I1N1Z0V0C0 A=FF B=10 X=8004 SP=A040 PC=E30B LDX A012
H0I1N1Z0V0C0 A=FF B=10 X=E124 SP=A040 PC=E30E CPX #E123
H0I1N0Z0V0C0 A=FF B=10 X=E124 SP=A040 PC=E311 BEQ E319
H0I1N0Z0V0C0 A=FF B=10 X=E124 SP=A040 PC=E313 LDX #E124
H0I1N1Z0V0C0 A=FF B=10 X=E124 SP=A040 PC=E316 STX A012
H0I1N1Z0V0C0 A=FF B=10 X=E124 SP=A042 PC=E319 RTS
H0I1N1Z0V0C0 A=FF B=10 X=E124 SP=A042 PC=E109 LDX #8004
H0I1N1Z0V0C0 A=FF B=10 X=8004 SP=A042 PC=E10C JSR E284
```

`LDX A012` reads the pointer into `X` (`E124`), `CPX #E123` compares it and clears `Z` because they differ, so `BEQ E319` falls through, `LDX #E124` / `STX A012` writes it back, and `RTS` returns to the reset code at `E109` — which immediately loads `X` with `8004`, the console ACIA's address, and calls the routine at `E284` that programs it. Eight steps in, you have followed the write you asked about all the way back into the reset sequence, and watched the console being pointed at next.

Saving state — `SNAPSHOT`, `RESTORE`

`SNAPSHOT` writes the machine's whole state — the CPU, the clock, and every board's registers, RAM and latches — to a file, and `RESTORE` reads it back into a machine of the same shape. So you can save the machine at a moment and return to it later.

Mark a spot, change something, and put it back:

```
swtpcsim> SNAPSHOT before.snap
snapshot written to before.snap
swtpcsim> DEPOSIT 100 00 00 00 00      trample four bytes
swtpcsim> RESTORE before.snap
restored from before.snap
swtpcsim> DUMP 100-103                they are back
0100  DE AD BE EF                      ....
```

It saves *state*, not *configuration* — the boards themselves are not in the file. `RESTORE` loads back into the machine you already have, and refuses a file that does not match its shape (the same boards, ids, and order), leaving the running machine untouched. Build that shape with the machine file or a `CONFIG LOAD` first, then restore into it.

Getting an assistant to do it — the MCP server

Everything in this document is a command you type. It is also a tool an **AI assistant** can call. Start the machine with `--mcp` instead of at a terminal —

```
$ swtpcsim <machine> --mcp
```

— and the same debugger is offered to an assistant as structured tools: it can set a breakpoint, run to it, read the registers, disassemble, step, dump memory, and read the bus recorder, on the *same* machine object the monitor drives. So instead of learning the commands, you can describe the symptom in a sentence and let the assistant work the machine for you. (The MCP server is covered in full in the **MCP server** chapter of the User Manual, including how to register it with clients other than the one below.)

Setting it up — the example is Claude Code. Two one-time steps. First, register the server so the assistant can reach the machine; run this in the directory that holds your machine file, so the relative paths resolve there:

```
$ claude mcp add swtpcsim -- swtpcsim <machine> --mcp
$ claude mcp list                # confirm it registered and is reachable
```

Second, hand the assistant its briefing. `DRIVING-WITH-AI.md` ships in the package; it is written for the assistant, not for you — it teaches these tools and the recipes for booting, building and debugging over

them. Copy it into the same directory, start `claude` there, and point the assistant at it before you give it a job:

```
$ cp /path/to/DRIVING-WITH-AI.md .  
$ claude  
> Read DRIVING-WITH-AI.md, then use the swtpcsim MCP tools for what follows.
```

From then on you talk to the assistant, not to the server.

What you say. Give the assistant the whole job in plain language, and name the tools so it drives the simulator rather than guessing:

Using the swtpcsim MCP tools, boot the machine to SWTBUG and single-step the reset code from `E0D0` — tell me where it programs the console ACIA and what it writes to the control register.

My loader hangs instead of reaching the FLEX prompt. Boot with the MCP tools, break where the DC-4 driver reads the WD179x status at `8018`, and single-step from there to tell me where it goes wrong.

Something is overwriting the scratchpad at `A043`. Set a memory-write breakpoint there, run until it trips, and show me the instruction and registers that did it.

What the assistant does with that. It works the same loop you would, one tool call at a time — boots the machine, sets the breakpoint, runs to it, disassembles and steps — and reports the instruction and registers back to you. You watch the reasoning and the fix; you type none of the commands.

When a typed tool does not reach. The assistant is not limited to the structured tools: it can run any monitor command and read the reply, so a conditional breakpoint (`BREAK E101 IF X==8004`) or an octal dump is one call away, exactly as it is for you.

Things to know about the bus

Not commands — facts about how the backplane behaves, the kind that turn a baffling reading into an expected one.

When **FF** is not data

FF is a perfectly good byte — the value -1 , a **STX** opcode, the top of a jump table — so most of the time it reads back as exactly what a board put there. But it is also what the bus gives you when *nobody answered*, and telling those two apart is worth a habit.

A read from an address no board decodes returns **FF.** That is not an error code and it is not a convention we invented — it is what a **floating bus** reads. Nobody is driving the data lines, they idle high, and the processor faithfully reads eight ones. Real hardware does exactly this.

It has a consequence worth knowing. The 6800 takes its vectors from the top of memory, so a machine whose ROM does not cover **FFF8 – FFFF** reads **FF FF** there and, on an interrupt or reset, jumps to **FFFF** — straight into the floating bus. That is not a fallback anybody coded; it is what the hardware does when the vectors are unmapped, and it is why a bare machine with no monitor ROM runs off into nowhere the instant it is started.

So when a read gives you **FF** you did not expect, ask **WHO**: it tells you whether a board answered with that byte or the bus floated because none did.